Linux 编程 实验一 实验报告

项目	内容
课程名称	Linux 编程
实验名称	实验一：Linux 基本命令与服务器搭建
姓名	张瀚文
学号	B24040514
班级	B240405
实验日期	2026-10-10
实验环境	VMware 虚拟机 · Ubuntu 22.04.5 LTS · 宿主 Windows

一、实验目的

1. 熟悉 Linux 操作系统环境，掌握 Linux 基本命令的使用。
2. 掌握 Linux 文件和目录的基本操作。
3. 掌握 Linux 用户管理与权限管理。
4. 掌握文件压缩、归档与文本查找命令。
5. 掌握 openssh、nginx、mysql 三类常见服务器的搭建与验证方法。

二、实验环境

项目	内容
虚拟机软件	VMware Workstation
操作系统	Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-138-generic x86_64)
虚拟机 IP	192.168.174.128
登录用户	azrael
内核版本	6.8.0-138-generic
远程连接工具	Xshell 8
数据库客户端	DBeaver 26.2.2
实验文档工具	VSCode + Markdown PDF 插件

> **关于学号信息的体现**：按实验要求"截图中必须体现学号和姓名"，本文档所有涉及目录名、文件名的命令，均使用了学号后四位 `0514`，并在每条截图的命令行末尾执行了 `B24040514 张瀚文`，确保信息可见。

三、实验内容与步骤

任务 1：目录操作

操作说明： 依次输入以下命令，观察输出。

```
pwd
cd ~
mkdir 0514
cd 0514
mkdir cs
ls
rmdir cs
ls
```

命令解释：

命令	作用
`pwd`	显示当前所在目录的绝对路径，结果为 `/home/azrael`
`cd ~`	切换到当前用户的家目录（`~` 即 `/home/用户名`）
`mkdir 0514`	创建名为学号后四位的目录
`cd 0514`	进入该目录，提示符变为 `~/0514$`
`mkdir cs`	在当前目录下创建子目录 `cs`
`ls`	列出当前目录内容，此时输出 `cs`
`rmdir cs`	删除**空目录** `cs`（`rmdir` 只能删空目录）
`ls`	再次列出，**无任何输出**，验证删除成功

实验结果： 第一次 `ls` 输出 `cs`；`rmdir cs` 后第二次 `ls` 无输出，目录已空，删除成功。

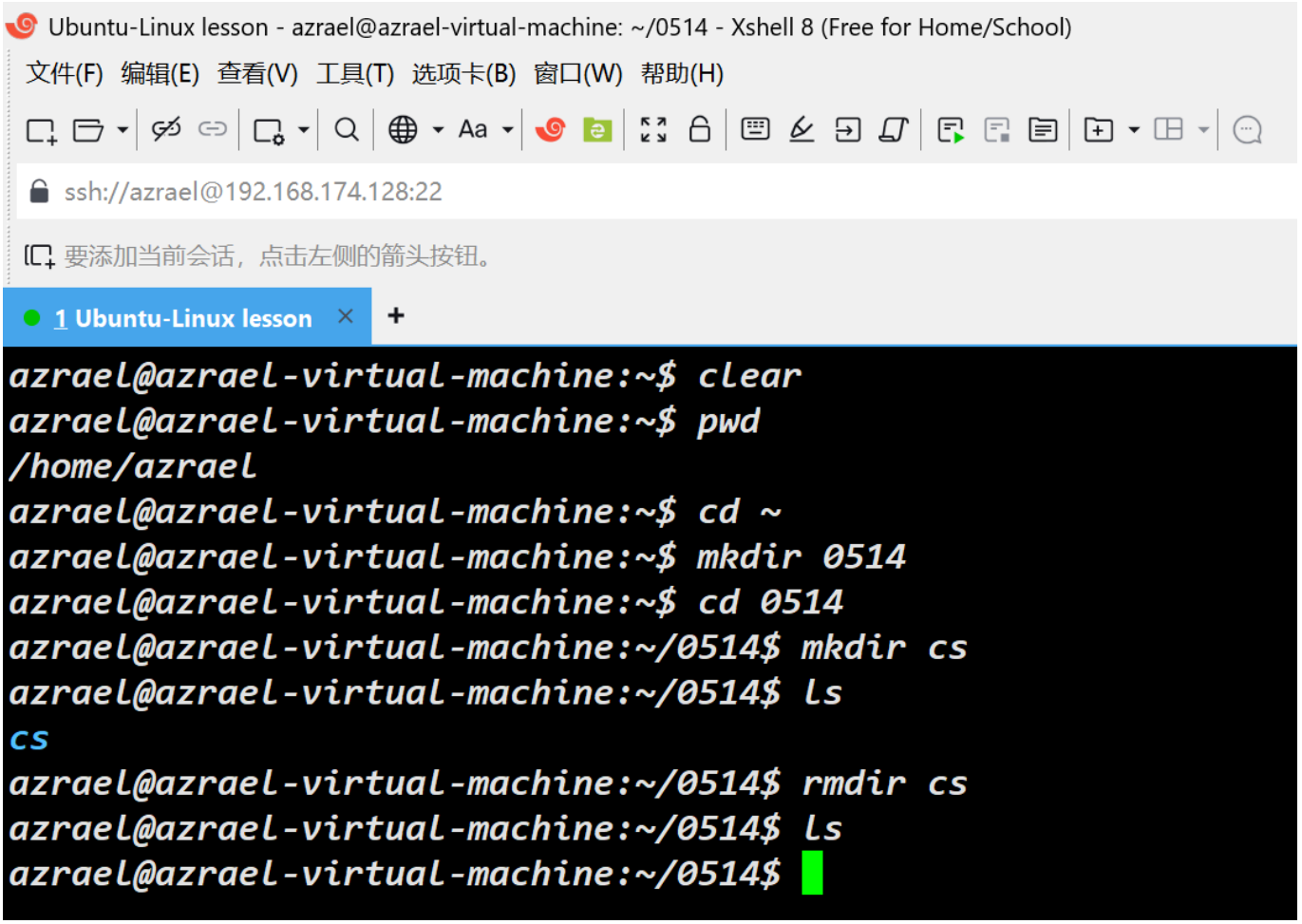


图 1-1　目录操作全过程（pwd 到 rmdir 后的 ls，路径中可见学号后四位 0514）

任务 2：文件操作

操作说明： 依次输入以下命令。

```
cd ~
touch 张瀚文.txt
ls
echo "hello world" > 0514.txt
cat 0514.txt
cp 张瀚文.txt 0514_copy.txt
mv 张瀚文.txt 0514_newname.txt
ls
```

命令解释：

命令	作用
`touch 张瀚文.txt`	创建一个空文件
`echo "hello world" > 0514.txt`	`>` 为**输出重定向**，把 `echo` 的内容写入文件（会覆盖原内容）
`cat 0514.txt`	查看文件内容，输出 `hello world`

命令	作用
`cp A B`	复制文件 A 为 B
`mv A B`	移动/重命名文件，此处为**重命名**

实验结果： `cat 0514.txt` 输出 `hello world`；最后的 `ls` 能看到三个文件（`0514.txt`、`0514_copy.txt`、`0514_newname.txt`），`张瀚文.txt` 已通过 `mv` 改名为 `0514_newname.txt`。

> **说明**：教材原文此处的 `cat` 命令写作 `last_4_number_of_your_studentID.txt`（少了 `s`），与上面创建的文件名不一致，应为笔误。本文档按实际可执行的写法记录。

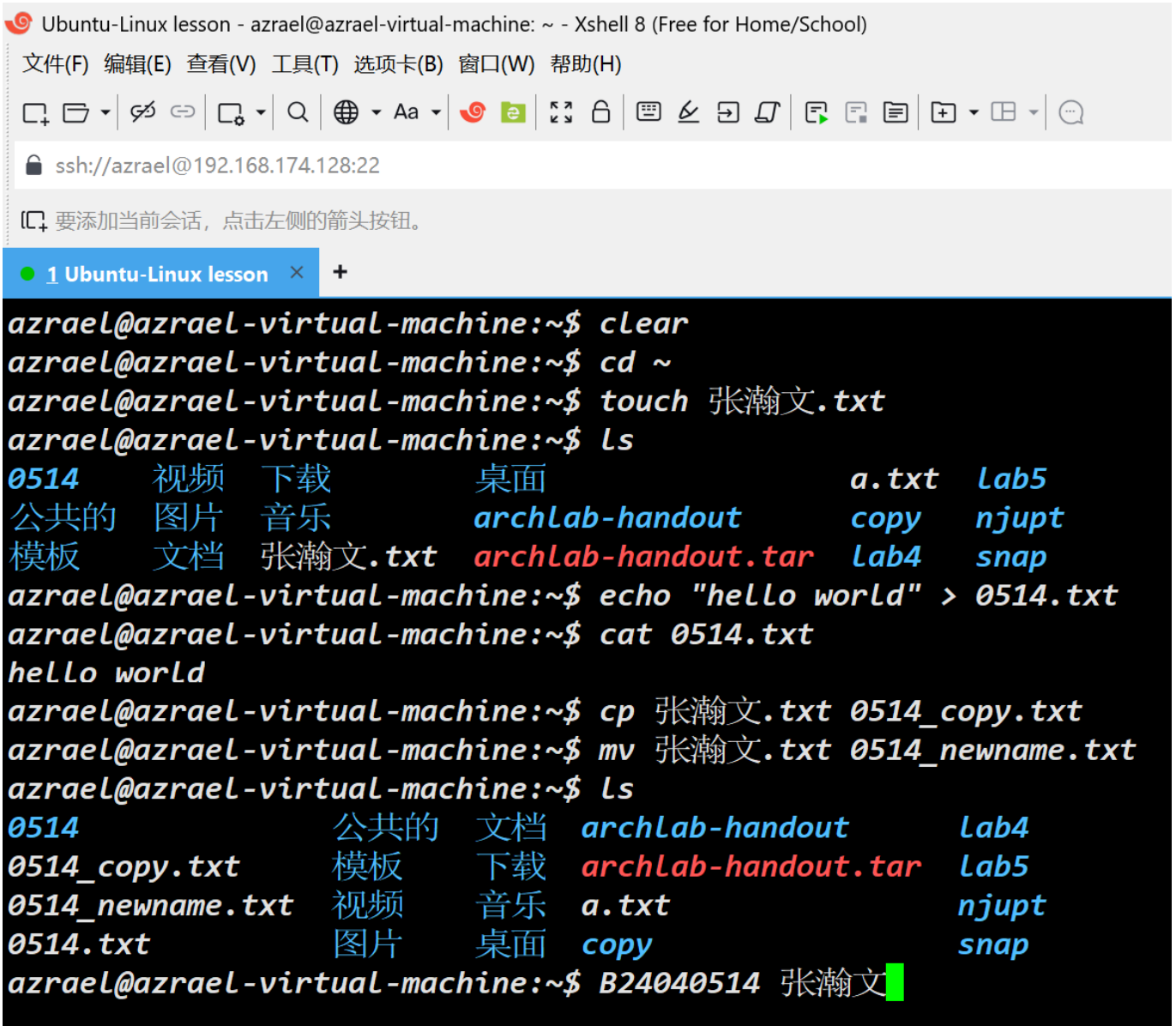


图 1-2　文件操作全过程（文件名中同时体现学号 0514 与姓名）

任务 3：用户管理

操作说明： 依次输入以下命令。

```
whoami
sudo su root
```

> 提示 `[sudo] azrael 的密码:`，输入**当前登录用户的密码**（Ubuntu 默认没有 root 密码，也不允许 root 直接登录）。

```
groupadd student
whoami
adduser ny0514
usermod -g student ny0514
```

命令解释：

命令	作用
`whoami`	显示当前用户名，首次执行输出 `azrael`
`sudo su root`	提权切换到 root，提示符由 `$` 变为 `#`，`whoami` 输出 `root`
`groupadd student`	新建用户组 `student`
`adduser ny0514`	新建用户（**交互式**，见下方说明）
`usermod -g student ny0514`	把用户 `ny0514` 的**主组**改为 `student`

> ⚠️ **本实验遇到的一个问题：用户名不能用大写。** 教材写的是 `adduser NY0101`（`NY` 大写）。但在 Ubuntu 22.04 上执行会报错：
>
> ```
> adduser: 请输入匹配 NAME_REGEX[_SYSTEM] 配置选项配置的正则表达式的用户名。
> ```
>
> **原因**：Ubuntu 的 `/etc/adduser.conf` 中 `NAME_REGEX` 默认只允许小写字母开头的用户名，大写字母不被接受。
>
> **解决**：改用全小写用户名 `adduser ny0514`，其余步骤（加入 student 组等）完全一致。这也是 Linux 用户名的通行惯例——**用户名习惯上全用小写**。

> `adduser` **会依次询问：**
>
> - `新的密码:` → 输入新用户密码（输入时**不显示**，属正常）
> - `重新输入新的密码:` → 再输一遍
> - `全名 []:`、`房间号码 []:`、`工作电话 []:` 等 → **全部直接回车跳过**
> - `这些信息是否正确？ [Y/n]` → 输入 `y` 回车

切换到新用户并验证归属：

```
su ny0514
cd ~
touch ny0514.txt
ls -l
```

命令解释：

命令	作用
`su ny0514`	切换到新用户（从 root 切换**无需密码**）
`cd ~`	进入新用户家目录 `/home/ny0514`
`touch ny0514.txt`	建文件用于验证归属
`ls -l`	长格式列出，**关键看第 3、4 列**

实验结果：

```
-rw-r--r-- 1 ny0514 student 0 10月 10 16:16 ny0514.txt
```

属主为 `ny0514`，属组为 `student`，证明用户创建成功且已正确加入 `student` 组。

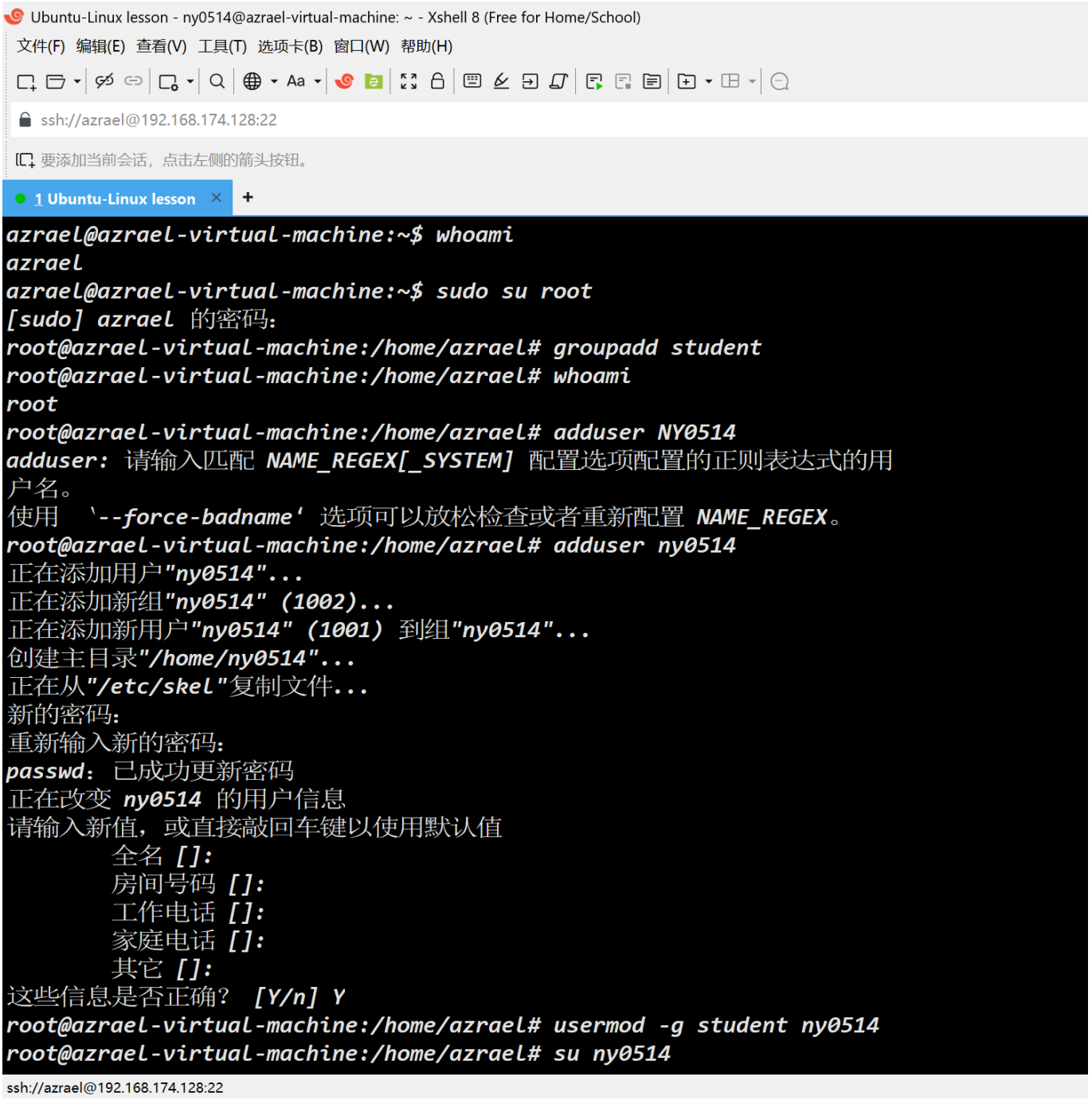


图 1-3　创建用户与用户组（可见大写用户名报错及改用小写的处理过程）

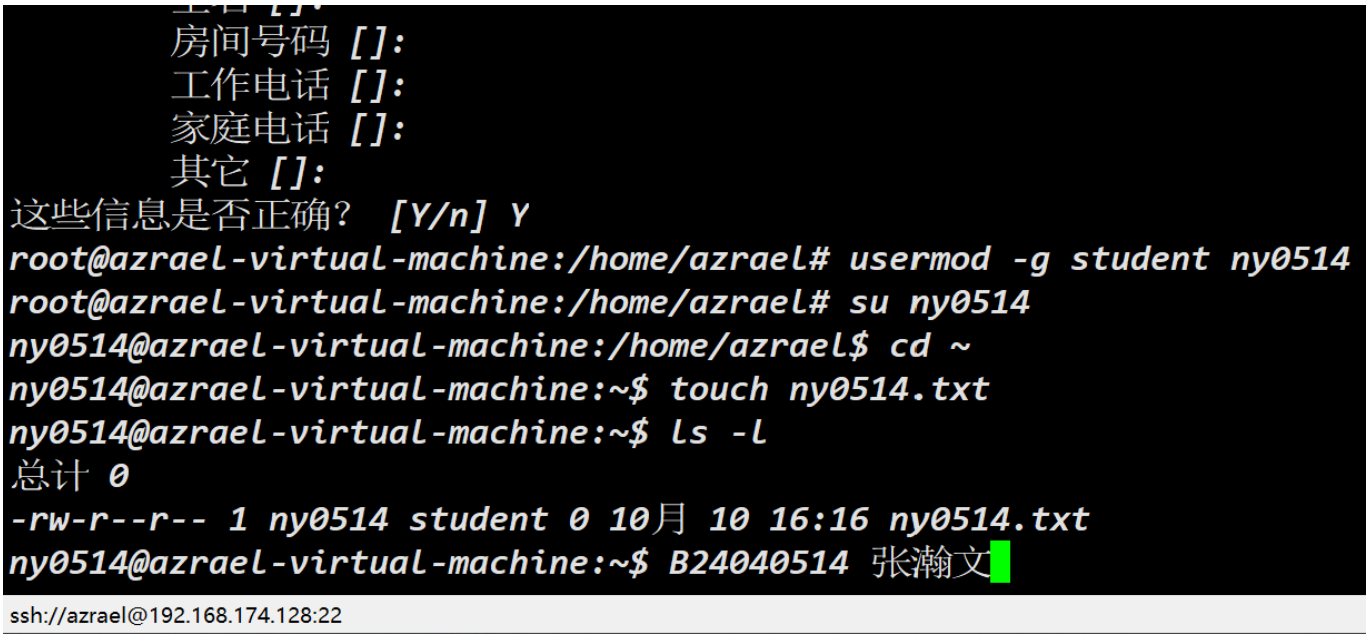


图 1-4　切换新用户后 `ls -l` 验证，属主 `ny0514`、属组 `student`

任务 4：权限管理

操作说明： 依次输入以下命令，**每输入一行，紧接着执行一次 `ls -l demo` 查看结果**。

```
touch demo
chmod 777 demo
chmod a-x demo
chmod go-w demo
chmod g+w demo
chmod a= demo
chmod +t demo
chmod u+s demo
chmod u+rx demo
chmod g+rws demo
```

权限变化对照表（与截图逐行核对）：

步骤	命令	`ls -l` 结果	说明
0	`touch demo`	`rw-r--r--`	新建文件默认权限 664（受 umask 022 影响）
1	`chmod 777 demo`	`rwxrwxrwx`	数字法：属主/属组/其他 都是 7
2	`chmod a-x demo`	`rw-rw-rw-`	`a` = all，去掉所有执行位
3	`chmod go-w demo`	`rw-r--r--`	`go` = group+other，去掉写权限
4	`chmod g+w demo`	`rw-rw-r--`	给属组加回写权限
5	`chmod a= demo`	`---------`	`=` 表示**清空**该类所有权限
6	`chmod +t demo`	`--------T`	加**粘滞位** sticky；无执行位时显示大写 `T`

步骤	命令	`ls -l` 结果	说明
7	`chmod u+s demo`	`--S-----T`	加 **SUID**（属主）；无执行位时显示大写 `S`
8	`chmod u+rx demo`	`r-s-----T`	属主补 `r`、`x`，SUID 因有 `x` 变为小写 `s`
9	`chmod g+rws demo`	`r-srwS--T`	属组加 `r`、`w`、`s`(SGID)。注意 `s` **不包含** `x`，故显示大写 `S`

> **要点理解：**
>
> - 字母含义：`r` 读、`w` 写、`x` 执行；`s` = SUID/SGID（有执行位显示小写 `s`，无则大写 `S`）；`t/T` = 粘滞位（同理大小写）。
> - 数字含义：`r=4, w=2, x=1`，三位依次为 属主/属组/其他。如 `777` = 三者都是 `4+2+1`。
> - **`chmod g+rws` 是本题的关键陷阱**：`rw` 之外的 `s` 只设置 SGID，**并不会授予执行位**，所以结果是大写 `S` 而不是小写 `s`。这正好说明"大写 S/T 表示对应特殊位已设置但执行位缺失"。

实验结果： 9 次 `chmod` 后 `ls -l` 的输出与上表**逐行完全一致**。

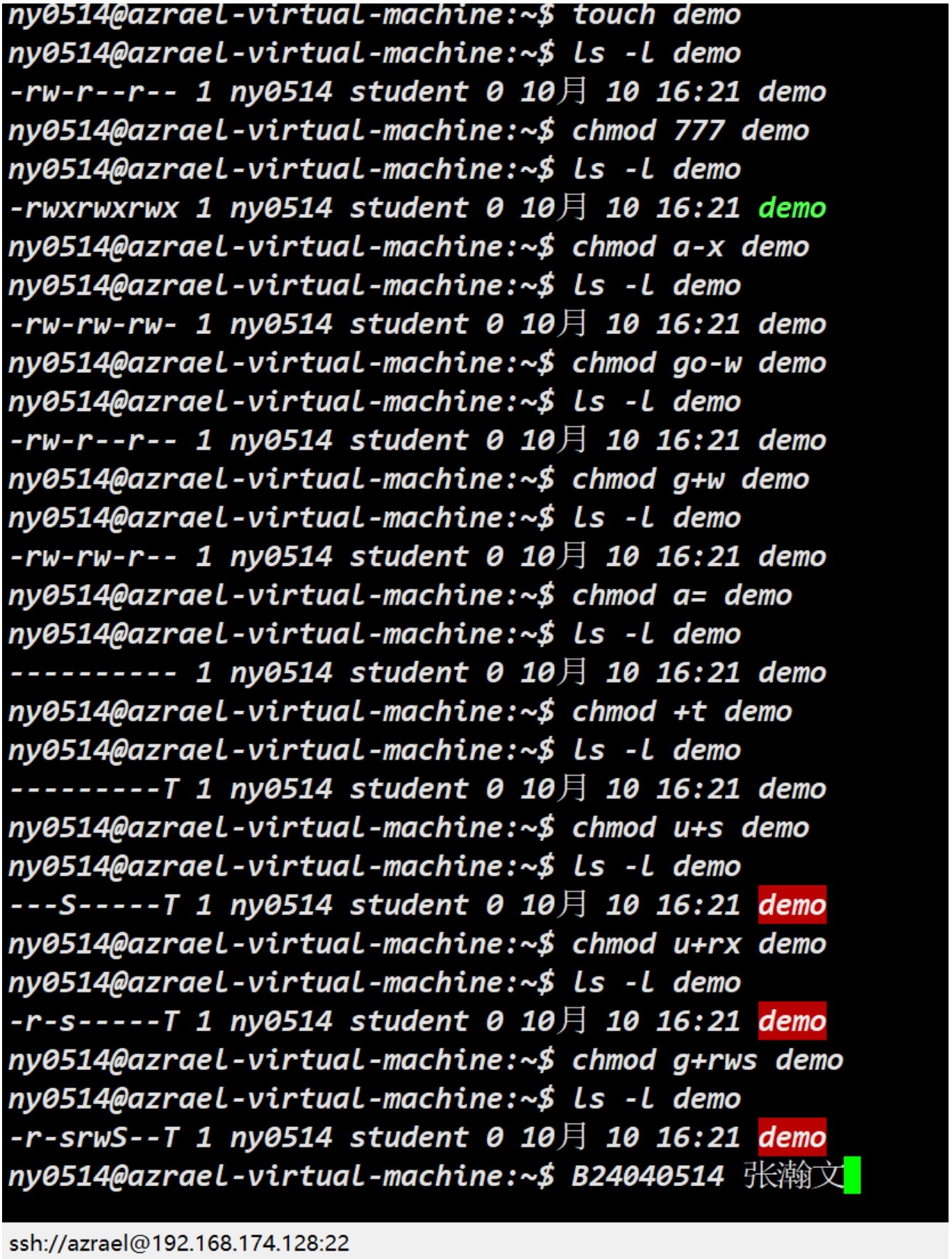


图 1-5　权限管理全过程，每条 `chmod` 后紧跟 `ls -l demo`

任务 5：其他管理（打包压缩 + 文本查找）

5.1 打包与解包

```
cd ~
mkdir others
cd others
touch file1 file2
cd ~
tar -czvf backup.tar.gz others
ls -l
tar -xzvf backup.tar.gz
ls -l
```

命令解释：

命令	作用
`cd ~`	**打包前必须回到被压缩目录的上一级**，否则 tar 找不到 `others` 会报 `Cannot stat`
`tar -czvf backup.tar.gz others`	`c` 创建归档、`z` gzip 压缩、`v` 显示过程、`f` 指定文件名
`tar -xzvf backup.tar.gz`	`x` 解包，其余参数同上

实验结果： 打包输出 `others/`、`others/file2`、`others/file1`，`ls -l` 可见 `backup.tar.gz` 大小 154 字节；解包时同样逐条打印文件名，`others` 目录被完整还原。

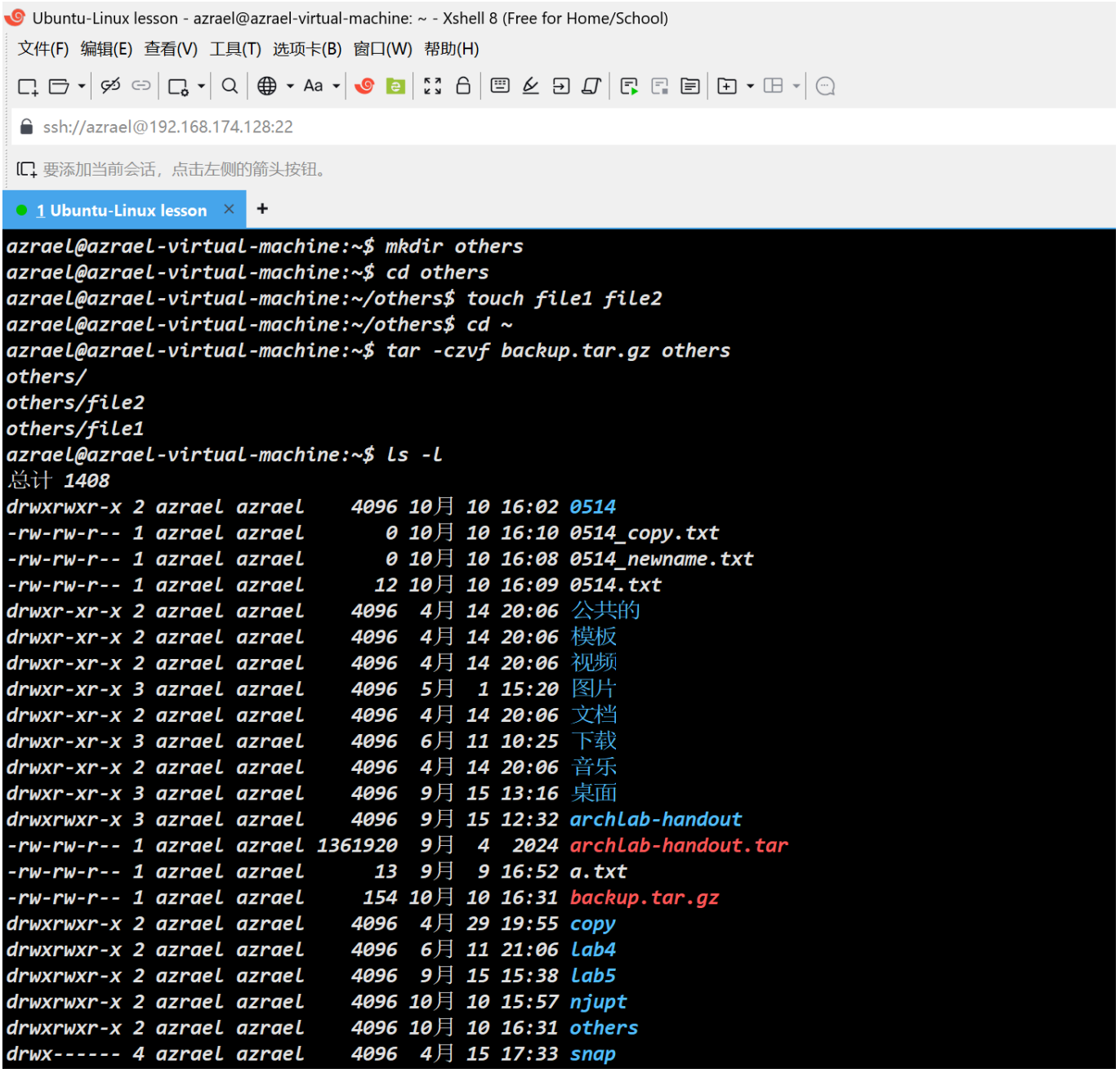


图 1-6　`tar -czvf` 打包过程及结果（`-v` 参数逐条列出被打包的文件）

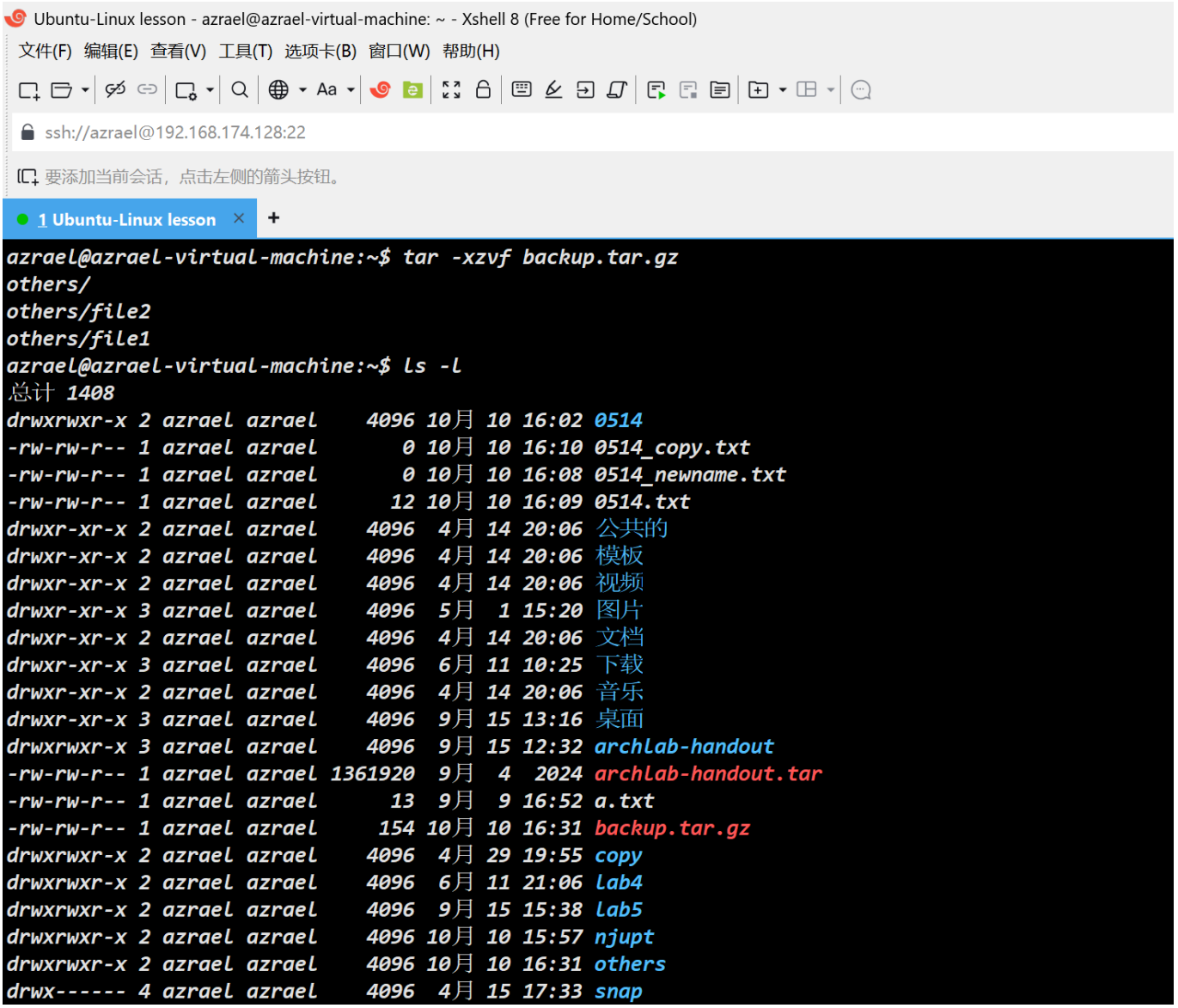


图 1-7 `tar -xzvf` 解包过程及结果

5.2 创建文本文件 textfile

```
cd ~
cat > textfile <<'EOF'
no, I study in njupt.
this line ends with 00
this line ends with .00
I am 25 years old.
He is 15. It is 16.00
wy is the short of my name.
EOF
```

> 使用**重定向 + heredoc**（`<<'EOF' ... EOF`）一次性写入多行内容，比用 vi 逐行输入更高效。结束符 `EOF` 必须**单独一行、顶格书写**。

5.3 grep 文本查找

```
grep '^n' textfile
grep '\.00$' textfile
grep '5\..' textfile
grep '^[wy]' textfile
```

正则符号与实验结果：

命令	正则含义	匹配结果
`grep '^n' textfile`	`^` 行首，找出以 `n` 开头的行	`no, I study in njupt.`
`grep '\.00$' textfile`	`\.` 转义的点号、`$` 行尾，找以 `.00` 结尾的行	`this line ends with .00` `He is 15. It is 16.00`
`grep '5\..' textfile`	数字 `5` + 一个 `.` + **任意一个字符**	`He is 15. It is 16.00`
`grep '^[wy]' textfile`	`^[wy]`，以 `w` 或 `y` 开头的行	`wy is the short of my name.`

> **逐条分析：**
>
> - **第 2 条**：`this line ends with 00` 虽然以 `00` 结尾，但**缺少小数点**，因此不匹配——这正是本题要考察的转义点号的意义。
> - **第 3 条**：`I am 25 years old.` 中的 `5` 后面跟的是**空格**而非 `.`，故不匹配；只有 `He is 15. It is 16.00` 满足"5 后紧跟点号"。
> - **第 4 条**：文本中没有以 `y` 开头的行，故只匹配到 `wy...` 这一行。

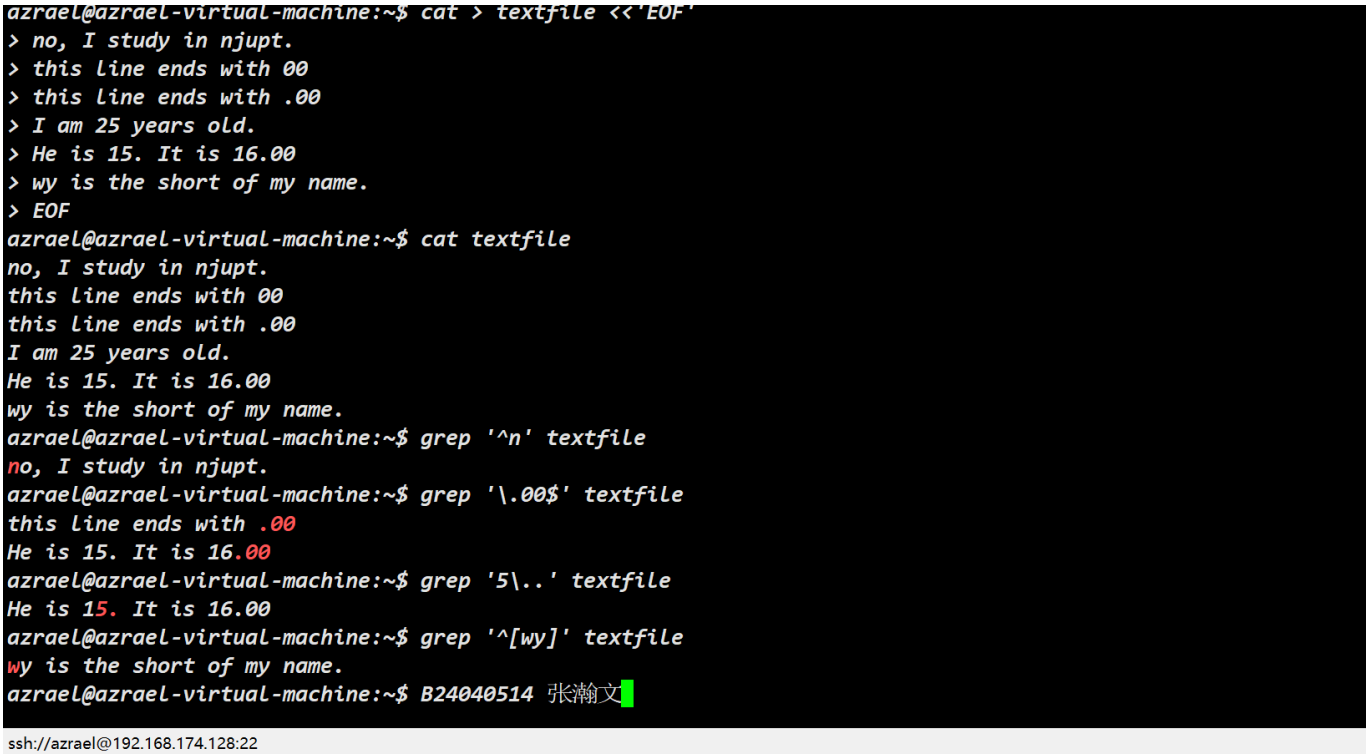


图 1-8　textfile 文件内容及四条 grep 命令的执行结果

四、服务器搭建

4.1 第一个服务器：openssh-server

目标： 搭建 SSH 服务器，允许从 Windows 主机远程访问虚拟机。

步骤 1：安装并启动服务

```
sudo apt install openssh-server
sudo systemctl enable --now ssh
systemctl status ssh
```

步骤 2：查询虚拟机 IP 地址

```
ifconfig
```

实验结果：

- `systemctl status ssh` 显示 **`Active: active (running)`**，`Loaded` 一行为 `enabled`（开机自启），日志中可见 `Server listening on 0.0.0.0 port 22`，说明服务已在所有网卡上监听 22 端口。
- `ifconfig` 输出中 `ens33` 网卡的 `inet` 地址为 **192.168.174.128**，即虚拟机的 IP。

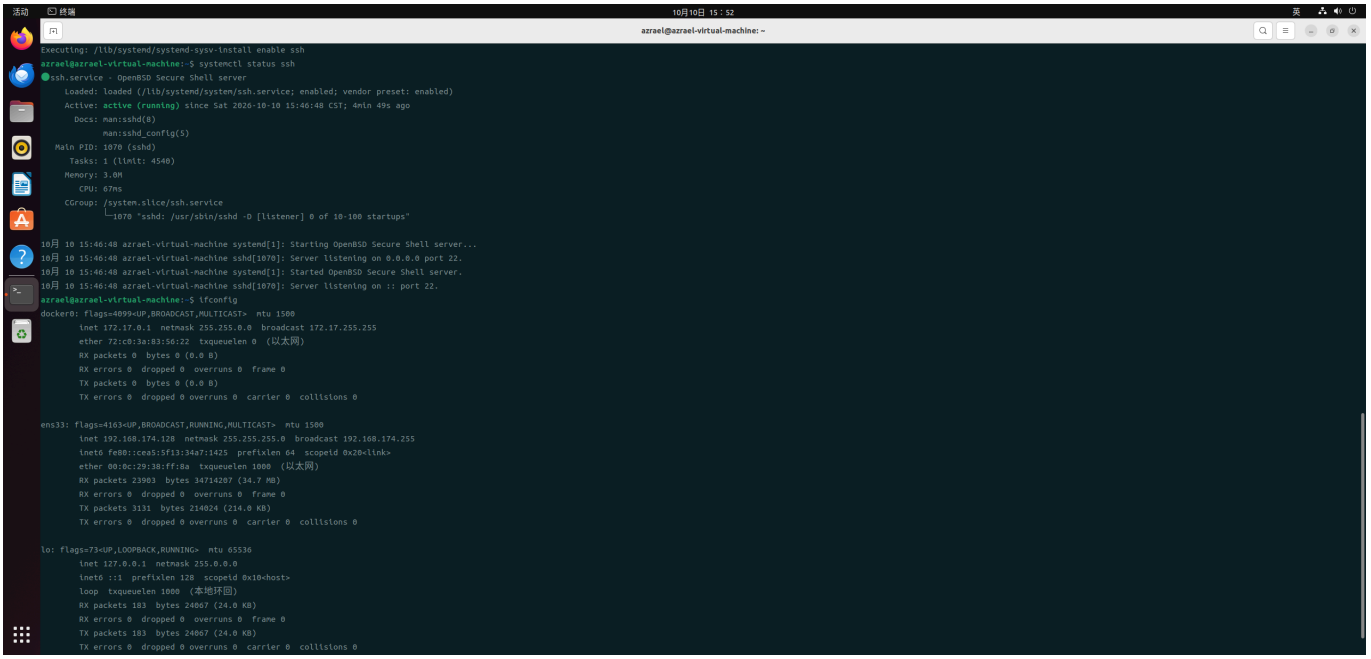


图 2-1　openssh-server 安装与运行状态（`active (running)`），以及 `ifconfig` 查询到的 IP 地址

步骤 3：使用远程工具连接虚拟机

在 Windows 上使用 **Xshell 8** 新建 SSH 会话，主机填上一步查到的 `192.168.174.128`，端口 `22`，登录用户 `azrael`。连接成功后终端显示 Ubuntu 22.04.5 LTS 欢迎信息，并提示上次登录来源为 `192.168.174.1`。**此后所有操作均在 Windows 端的终端窗口中进行。**

步骤 4：依次执行命令

```
pwd
ls
mkdir njupt
cd njupt
touch hello.txt
ls
echo "I love njupt" > hello.txt
cat hello.txt
```

实验结果： `pwd` 输出 `/home/azrael`；`cd njupt` 后 `ls` 为空；`touch hello.txt` 后 `ls` 显示 `hello.txt`；最后 `cat` 输出 `I love njupt`，说明远程操作与本地操作效果完全一致。

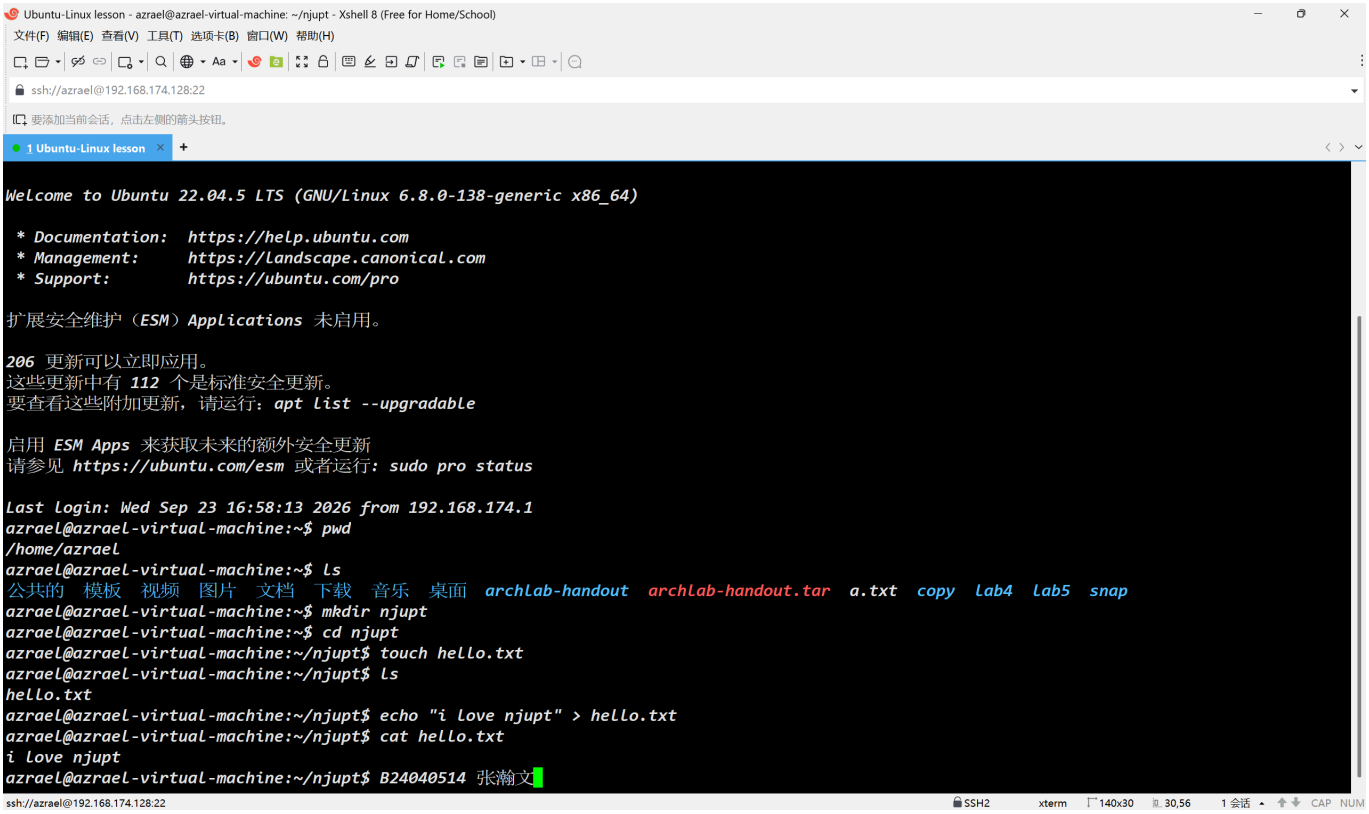


图 2-2　通过 Xshell 远程连接成功，并完成实验要求的文件操作命令

4.2 第二个服务器：web 服务器 nginx

目标： 搭建 nginx，实现从 Windows 浏览器访问虚拟机网页。

步骤 1：安装 nginx

```
sudo apt install -y nginx
systemctl status nginx
```

> 安装后 nginx 会自动启动。`systemctl status nginx` 应显示 `active (running)`。

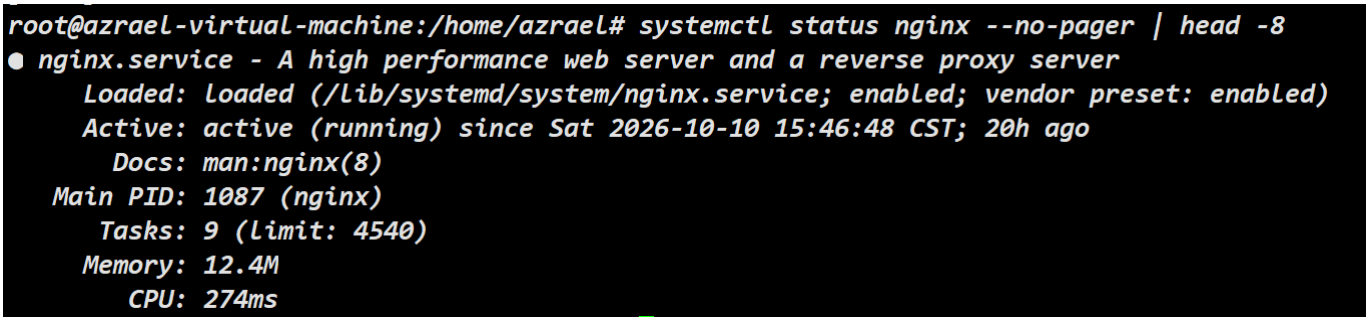


图 2-3　nginx 服务运行状态：`Active: active (running)`，`Loaded` 显示 `enabled`（已设开机自启）

步骤 2：查询虚拟机 IP

```
ifconfig
```

> IP 与 SSH 相同，均为 `192.168.174.128`。

步骤 3：在 Windows 浏览器中访问

打开 Windows 上的浏览器，地址栏输入 `http://192.168.174.128`，能看到 nginx 的默认欢迎页面即说明服务可用。（浏览器提示"不安全"属正常，因为使用 http 而非 https。）

步骤 4：修改网页内容

```
sudo su root
cd /var/www/html
vim index.nginx-debian.html
```

将内容修改为：

```
<html><body>hello, njupt</body></html>
```

> **说明**：nginx 的默认站点根目录为 `/var/www/html`，其默认索引文件中 `index index.html index.htm index.nginx-debian.html;` 决定优先级。 本实验直接修改了**默认索引页 `index.nginx-debian.html`**；若新建 `index.html`，则其优先级更高，会覆盖默认页的效果。两种做法都能达到"自定义首页"的目的。

实验结果： 保存后在 Windows 浏览器刷新，页面显示 **`hello,njupt`**，说明自定义首页生效。

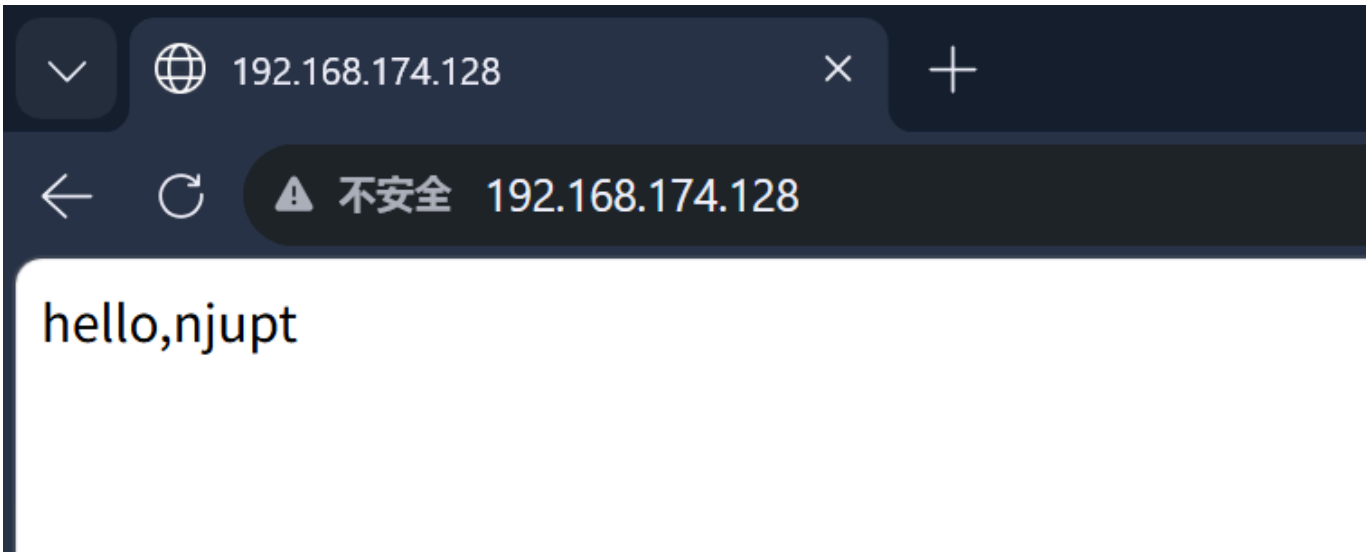


图 2-4　在 Windows 浏览器中访问 `http://192.168.174.128`，显示自定义内容 `hello,njupt`

4.3 第三个服务器：mysql 服务器

目标： 搭建 MySQL 数据库服务器，建库建用户，并用图形化工具连接验证。

步骤 1：切换管理员

```
sudo su root
```

步骤 2：安装 mysql-server

```
apt install mysql-server
```

步骤 3：查看服务状态

```
systemctl status mysql
```

> **注意**：此命令输出较长，会进入分页器，需要按 `q` 退出。若不想进分页器，可加 `--no-pager` 参数。

```
root@azrael-virtual-machine:/home/azrael# systemctl status mysql --no-pager | head -8      # → 补 3-1
● mysql.service - MySQL Community Server
     Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
     Active: active (running) since Sat 2026-10-10 18:19:56 CST; 17h ago
    Process: 26594 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Main PID: 26602 (mysqld)
     Status: "Server is operational"
      Tasks: 40 (limit: 4540)
     Memory: 375.0M
```

图 3-1　MySQL 服务运行状态：`Active: active (running)`，`Status: "Server is operational"`

步骤 4：进入 MySQL 并建库建用户

```
mysql
```

> **为何无需输入密码**：Ubuntu 下 MySQL 的 `root` 用户默认使用 `auth_socket` 认证插件，只要当前是系统 root 用户即可直接登录。**因此必须先执行 `sudo su root`**，否则会报 `Access denied`。

进入后提示符变为 `mysql>`，依次执行：

```
CREATE DATABASE mydb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'myuser'@'localhost' IDENTIFIED BY '111111';
GRANT ALL PRIVILEGES ON mydb.* TO 'myuser'@'localhost';
FLUSH PRIVILEGES;
SHOW DATABASES;
exit
```

SQL 语句解释：

语句	作用
`CREATE DATABASE mydb ...`	创建数据库 `mydb`，字符集 `utf8mb4`（完整 UTF-8，支持 emoji）、排序规则 `utf8mb4_unicode_ci`
`CREATE USER 'myuser'@'localhost' ...`	创建用户 `myuser`，密码 `111111`，**仅允许本机登录**
`GRANT ALL PRIVILEGES ON mydb.* TO ...`	把 `mydb` 库的全部权限授予该用户，体现**最小权限原则**
`FLUSH PRIVILEGES;`	刷新权限表。通过 `GRANT` 命令授权本已自动生效，此条用于确保手动修改权限表后的改动也立即生效
`SHOW DATABASES;`	列出所有数据库，应能看到 `mydb`

> **补充说明**：MySQL 8 起不再支持用 `GRANT ... IDENTIFIED BY` 隐式创建用户，因此"创建用户"与"授权"必须拆成两条语句。 **关于 `utf8mb4`**：MySQL 中的 `utf8` 是残缺实现（最多 3 字节），无法存储 emoji 和部分生僻汉字，`utf8mb4` 才是真正的 UTF-8。

实验结果： `SHOW DATABASES;` 输出中包含 `mydb`，与 `information_schema`、`mysql`、`performance_schema`、`sys` 四个系统库并列，共 5 行。

> 本次执行时 `CREATE DATABASE` 曾返回 `ERROR 1007: Can't create database 'mydb'; database exists`，原因是此前已执行过一次建库操作；数据库已存在不影响后续授权与验证，`SHOW DATABASES;` 的输出证实 `mydb` 存在。

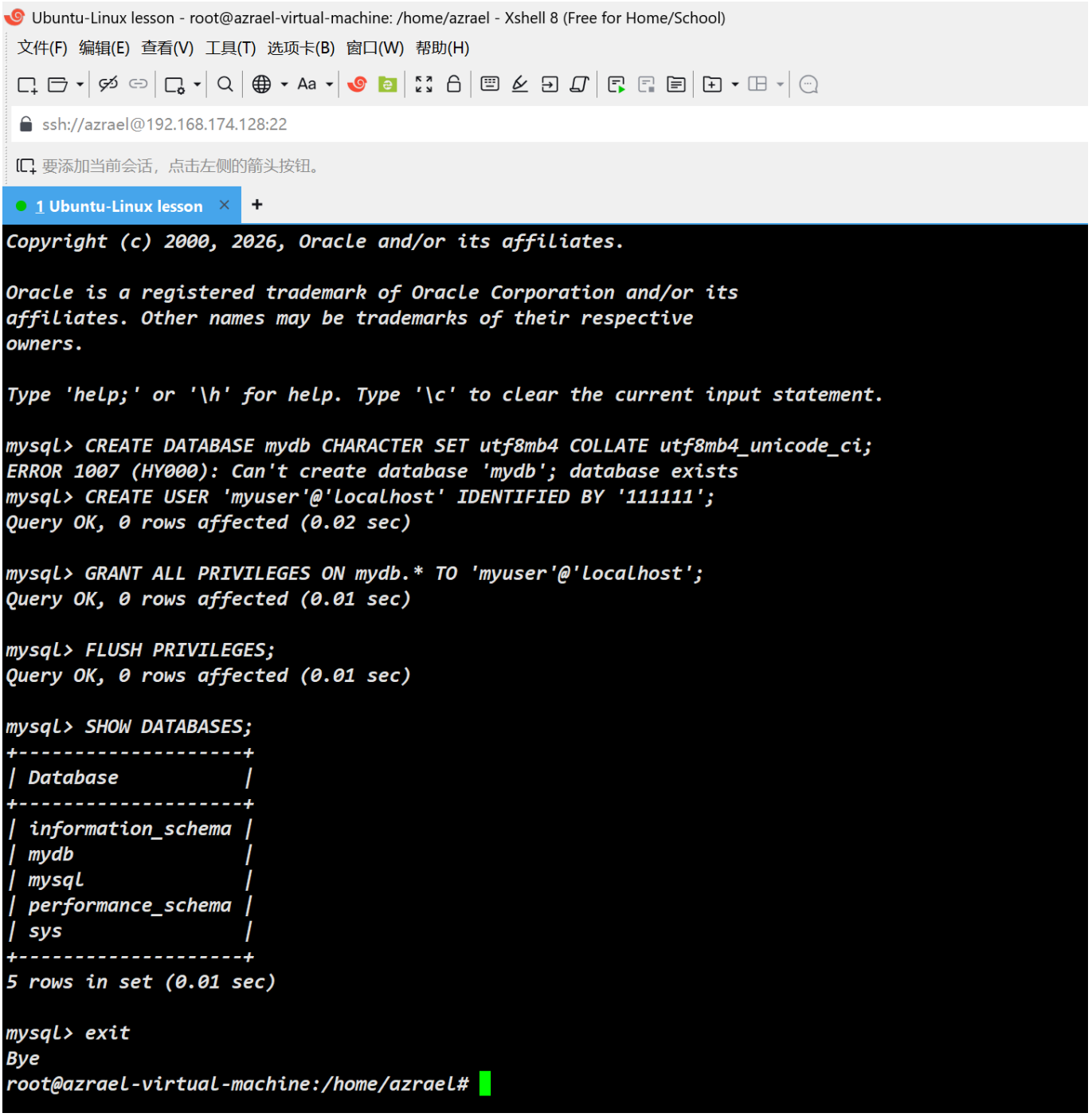


图 3-2　进入 `mysql>` 后执行建库、建用户、授权、刷新权限及查询数据库列表

步骤 4.5：用新建用户验证登录

```
mysql -u myuser -p
```

输入密码 `111111` 后成功进入。

```
SHOW DATABASES;
```

实验结果： 以 `myuser` 身份执行 `SHOW DATABASES;` **只能看到 3 个库**（`information_schema`、`mydb`、`performance_schema`），**看不到 `mysql` 和 `sys` 系统库**——这正好证明前面的最小权限授权生效：该用户只能访问 `mydb`。

```
root@azrael-virtual-machine:/home/azrael# mysql -u myuser -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.46-0ubuntu0.22.04.4 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mydb               |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)

mysql> exit
Bye
```

图 3-3　使用 `myuser` 登录成功，且只能看到其有权限的数据库

步骤 5：使用图形化工具 DBeaver 连接

在 Windows 端安装 **DBeaver 26.2.2**，新建 MySQL 连接，参数如下：

参数	值
Host	`192.168.174.128`
Port	`3306`
Database	`mydb`
Username	`myuser`
Password	`111111`

点 `Test Connection`，返回 **`Connected (8178 ms)`**，服务器版本 `MySQL 8.0.46-0ubuntu0.22.04.4`，驱动为 `mysql-connector-j-8.2.0`，连接成功。

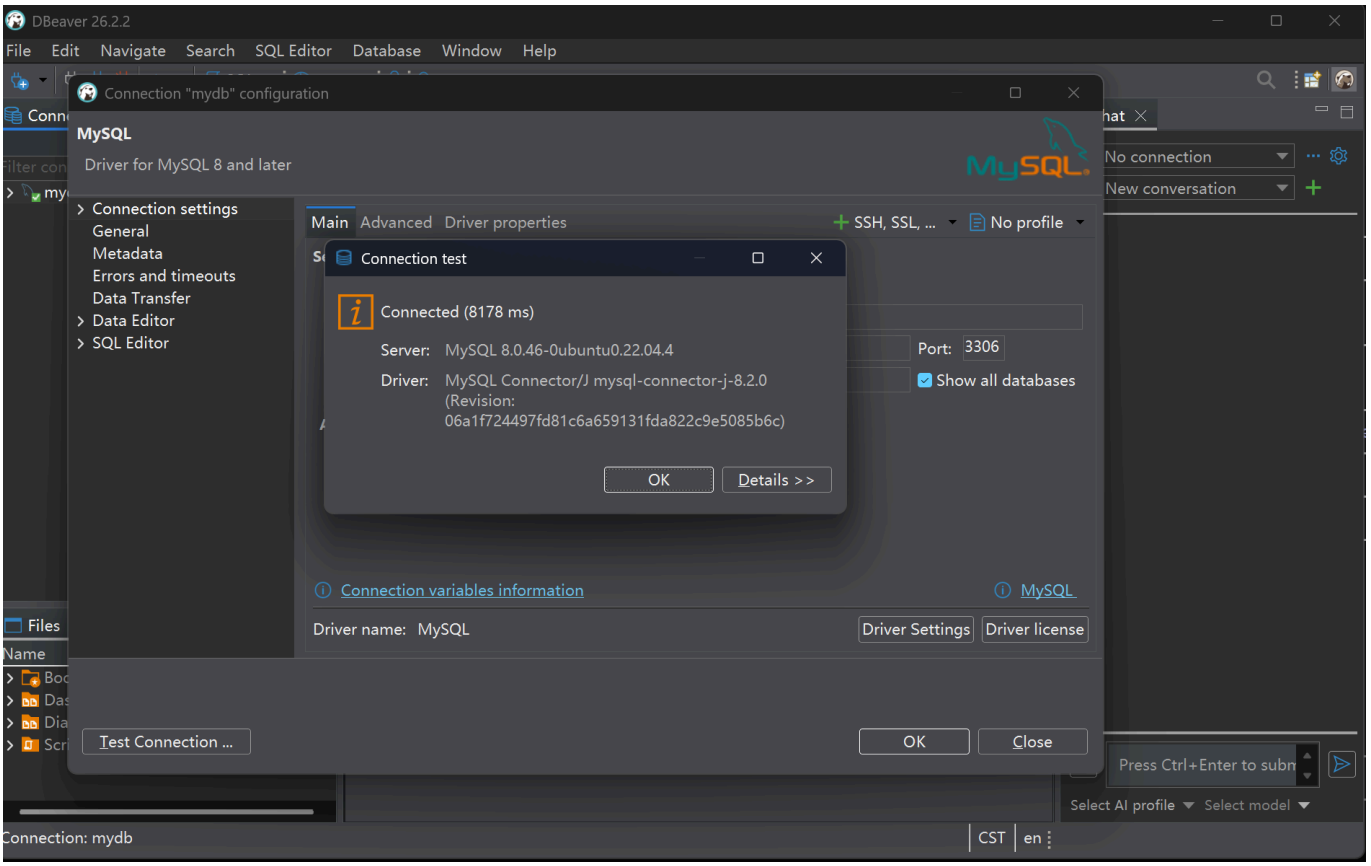


图 3-4　DBeaver 测试连接成功（Connected），连接对象为虚拟机上的 mydb

> **关于本步的网络处理**：详见"五、实验遇到的问题与解决"中的问题 3。

五、实验遇到的问题与解决

问题 1：adduser 拒绝大写用户名

现象：按教材执行 `adduser NY0514` 时报错：

```
adduser: 请输入匹配 NAME_REGEX[_SYSTEM] 配置选项配置的正则表达式的用户名。
```

原因：Ubuntu 的 `/etc/adduser.conf` 中 `NAME_REGEX` 默认只接受小写字母开头的用户名，大写字母不符合规则。

解决：改用小写用户名 `adduser ny0514`，后续 `usermod`、`su`、`touch ny0514.txt` 等命令同步使用小写。这也符合 Linux 用户名全小写的通行惯例。

问题 2：systemctl status 命令"卡住"

现象：执行 `systemctl status mysql` 后终端停止响应，看起来像死机。

原因：该命令输出较长，systemd 自动调用 `less` 分页器显示。**分页器在等待按键，并非卡死**（屏幕左下角会出现 `:` 提示）。

解决：按 `q` 退出分页器即可。若希望输出直接铺在屏幕上便于截图，可加 `--no-pager` 参数：

```
systemctl status mysql --no-pager
```

问题 3：虚拟机无法访问外网，导致 snap 商店与 DBeaver 下载失败

现象：执行 `snap install dbeaver-ce --classic` 提示无法连接 snap 商店；`ping api.snapcraft.io` 报"域名解析出现暂时性错误"；进一步测试 `ping 223.5.5.5` 返回 `Destination Host Unreachable`。

排查过程：

1. `ip route` 显示默认网关为 `192.168.174.2`，网卡 `ens33` 已获得 IP `192.168.174.128`，**虚拟机内部路由配置正常**；
2. `cat /sys/class/net/ens33/operstate` 返回 `up`，说明虚拟网卡链路正常；
3. `arp -n` 是关键证据——同一网段的三个邻居中，`192.168.174.254`（DHCP 服务）和 `192.168.174.1`（宿主机虚拟网卡）**均正常应答**，唯独 **`192.168.174.2`（NAT 网关）显示 `(incomplete)`**。

结论：故障不在 Ubuntu 内，而是宿主机的 **VMware NAT 转发服务未运行**，导致虚拟机无法经 NAT 访问外网。但注意——**宿主机与虚拟机之间的直连通信是正常的**（`.1` 能应答即为证明）。

解决：改用**宿主机直连虚拟机数据库**的方案，绕开 NAT：

1. 在虚拟机上修改 MySQL 监听地址，由 `127.0.0.1` 改为 `0.0.0.0`（`/etc/mysql/mysql.conf.d/mysqld.cnf`）；
2. 创建允许远程登录的账号 `myuser'@'%'` 并授权；
3. 用 `ss -lntp | grep 3306` 确认已监听 `0.0.0.0:3306`；
4. 在 **Windows 端**安装 DBeaver，连接 `192.168.174.128:3306`，连接成功。

该方案同时完成了教材中"如果希望通过 windows 直接访问数据库"的选做要求。

问题 4：修改配置后 MySQL 服务无法启动

现象：修改 `bind-address` 后执行 `systemctl restart mysql` 失败：

```
Job for mysql.service failed because the control process exited with error code.
```

排查过程：

1. `systemctl status mysql` 与 `journalctl -xeu mysql.service` 输出的是 systemd 自身的注解信息，**未包含 MySQL 的错误原因**；
2. 查看 MySQL 自己的日志 `/var/log/mysql/error.log`，得到关键错误：

```
[ERROR] [MY-000067] [Server] unknown variable 'aa=a^;'.
```

3. 用 `grep -rn 'aa=a' /etc/mysql/` 定位到错误来源：`/etc/mysql/mysql.conf.d/mysqld.cnf` 第 17 行存在一行**无意义的非法内容 `aa=a^;#`**，导致 MySQL 解析配置失败，并因 `Restart=on-failure` 陷入每 1.5 秒一次的重启循环。

解决：

```
cp /etc/mysql/mysql.conf.d/mysqld.cnf ~/mysqld.cnf.before-fix
sed -i '/aa=a/d' /etc/mysql/mysql.conf.d/mysqld.cnf
systemctl start mysql
```

删除该行后服务恢复 `active (running)`，且 `mydb` 数据库与用户数据完好无损。

经验总结：

- **"时间上相邻"不等于"因果"**。本次故障发生在修改配置之后，但真正原因是配置文件中混入的非法行，与所修改的 `bind-address` 无关。先用 `grep -n 'bind'` 验证配置正确，才排除了这个错误假设。
- **服务启动失败时，应直接查阅服务自身的日志**，而不是只看 `systemctl status` 或 `journalctl` 的摘要。
- **修改配置文件应尽量使用 `sed` 等命令整行替换，避免手工编辑时误触键盘或输入法引入全角字符。**

六、实验小结

本次实验完成了以下工作：

1. 通过 `pwd`、`ls`、`mkdir`、`cd`、`rmdir` 等命令，掌握了 Linux 目录的基本操作；
2. 通过 `touch`、`echo` 重定向、`cat`、`cp`、`mv`，掌握了文件的新建、写入、查看、复制与重命名；
3. 通过 `groupadd`、`adduser`、`usermod`、`su`，掌握了用户与用户组的创建及归属管理，并实践了用户名命名规则的约束；
4. 通过 `chmod` 的符号法与数字法，理解了 Linux 权限模型，以及 SUID、SGID、粘滞位的含义与大小写显示规则；
5. 通过 `tar` 掌握了文件打包与解包，通过 `grep` 配合正则表达式掌握了文本查找；
6. 独立搭建了 **openssh、nginx、mysql** 三个服务器，并分别用 **Xshell 远程登录、浏览器访问、DBeaver 连接**三种方式完成验证；
7. 在排障过程中，实践了从"查看服务自身日志"到"逐层验证网络连通性（`ip route` → `arp -n` → `operstate`）"的定位方法，加深了对 Linux 系统与虚拟化网络的理解。

通过本次实验，我对 Linux 的命令行操作、权限机制、服务部署流程以及故障排查方法都有了直观且扎实的认识。

附录：截图清单

编号	内容	图片文件	状态
1-1	目录操作	实验-1.png	✅
1-2	文件操作	实验-2.png	✅
1-3	用户管理（创建用户并加入 student 组）	实验-3-1.png	✅
1-4	新用户 `ls -l` 验证归属	实验-3-2.png	✅
1-5	权限管理全过程	实验-4.png	✅

编号	内容	图片文件	状态
1-6	tar 打包	实验-5-1.png	✅
1-7	tar 解包	实验-5-2.png	✅
1-8	textfile 内容与 grep 四条命令	实验-5-3.png	✅
2-1	openssh 安装、运行状态与 IP	openssh服务器ubt截图.png	✅
2-2	Xshell 远程连接与文件操作	openssh服务器xshell截图.png	✅
2-3	nginx 服务运行状态	补2-3.png	✅
2-4	浏览器访问自定义页面	nginx服务器win端.png	✅
3-1	mysql 服务运行状态	补3-1.png	✅
3-2	建库、建用户、授权与 SHOW DATABASES	mysql服务器-1.png	✅
3-3	myuser 登录验证	mysql服务器-2.png	✅
3-4	DBeaver 连接成功	mysql连接成功.png	✅

完成度：16 / 16 ✅